



UML

Diagramme de classes

Diagramme de classes

Le diagramme de classes est considéré comme le plus important de la modélisation orientée objet, il est le seul obligatoire lors d'une telle modélisation.

Il permet de fournir une représentation abstraite des objets du système qui vont interagir pour réaliser les cas d'utilisation. Il est important de noter qu'un même objet peut très bien intervenir dans la réalisation de plusieurs cas d'utilisation.

Diagramme de classes

Le diagramme de classes permet de modéliser les classes du système et leurs relations indépendamment d'un langage de programmation particulier.

Modéliser la structure interne du système

Diagramme de classes d'Analyse (DCA)

- représenter les **classes métiers (modèle)** issues du Cahier de Charges
- **Exemple d'un système de gestion de commandes** : classes Client, Commande, Produit

Niveau Analyse

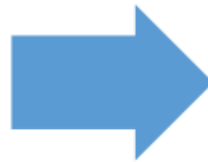


Diagramme de classes de Conception (DCC)

- Compléter par les **classes de Vue, de Contrôle, etc.**
- Exemple : classes pour les interfaces graphiques de l'application, classe Contrôleur pour contrôler la saisie des données, la sécurité, etc.

Niveau Conception

Structuration orienté objet



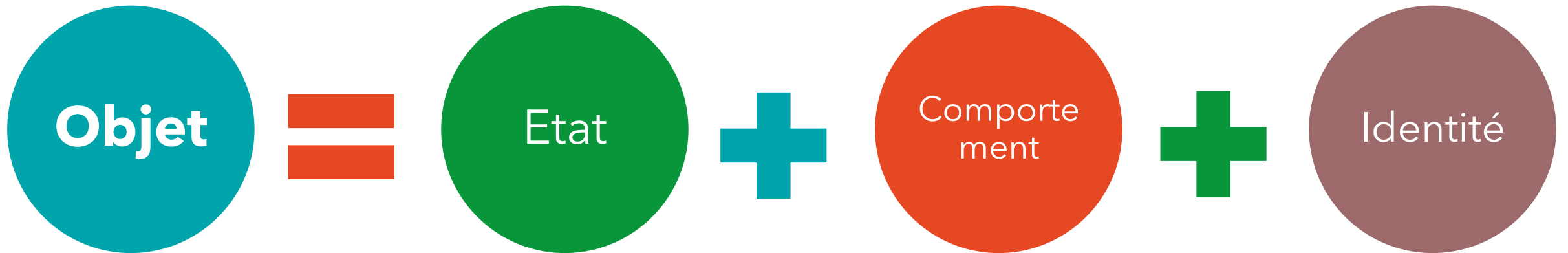
Qu'est-ce qu'une classe d'objet?

Une classe est une représentation abstraite d'un d'ensemble d'objets.

Elle contient les informations nécessaires à la construction de l'objet (c'est-à-dire la définition des attributs et des méthodes).

On dit qu'un objet est l'instance d'une classe.

Qu'est-ce qu'un objet?



Qu'est-ce qu'un objet?

Etat d'un objet

Ensemble de valeurs décrivant l'objet

Chaque valeur est associée à un attribut (propriété)

Les valeurs sont également des objets (→ liens entre objets)

Comportement d'un objet

Ensemble d'opérations que l'objet peut effectuer

Chaque opération est déclenchée par l'envoi d'un message (Appel d'une méthode)

Identité d'un objet

Permet de distinguer les objets indépendamment de leur état

Attribuée automatiquement à la création de l'objet

L'identité d'un objet ne peut être modifiée

Identité <> nom de la variable qui référence l'objet

Les classes

Classe avec instance

Classe concrète = regroupement d'objets (instances) possédant les mêmes caractéristiques

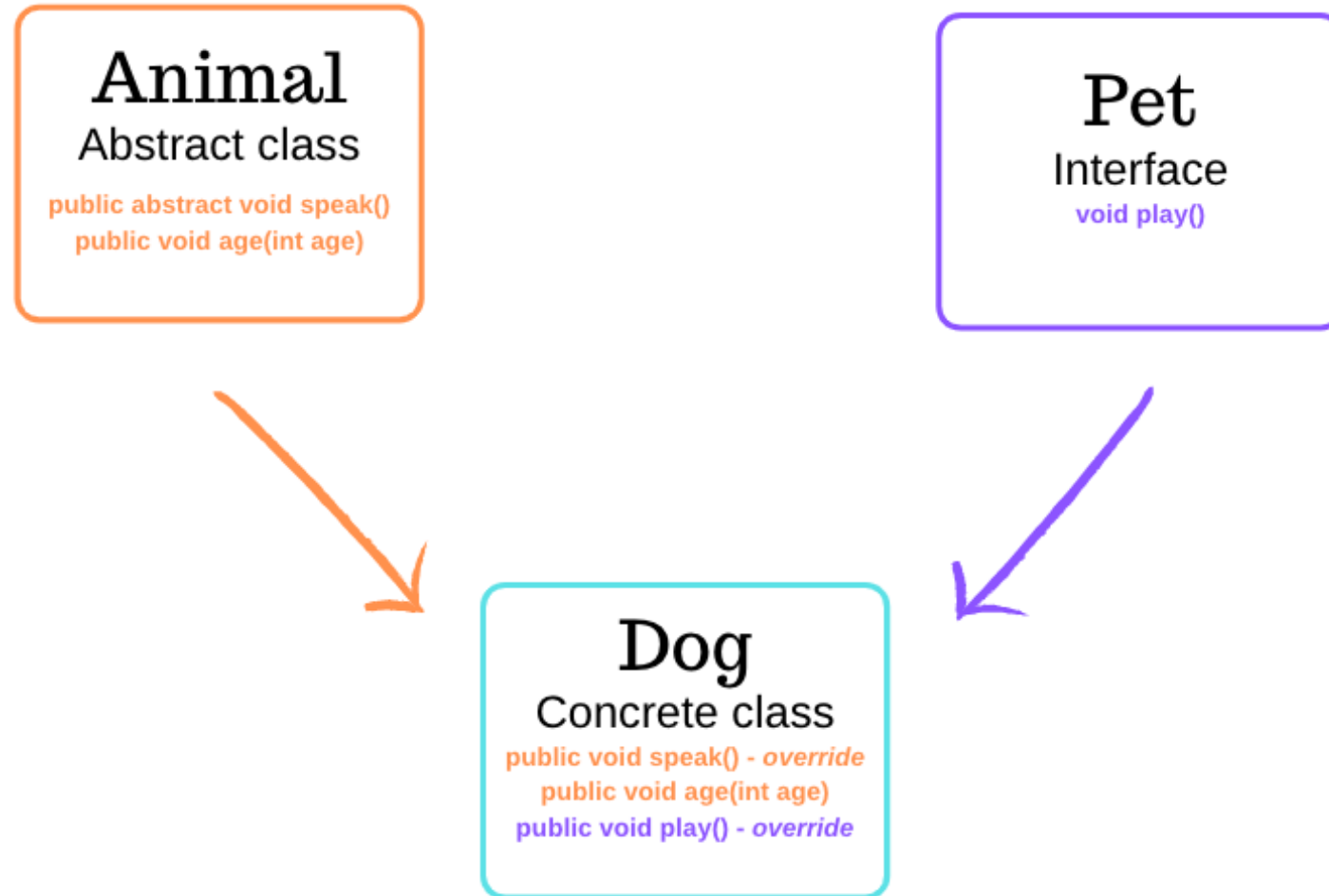
Toutes les instances d'une classe ont les mêmes attributs et opérations

Classes sans instance

Classes abstraites = certaines opérations peuvent être non définies

Interfaces : pas d'attribut et toutes les opérations sont non définies

Les classes



Représentation graphique d'une classe avec UML

Une classe est représentée par un rectangle (appelé aussi classeur) divisé en 3 compartiments :

- Le 1er compartiment contient le nom de la classe qui représente le type d'objet instancié.

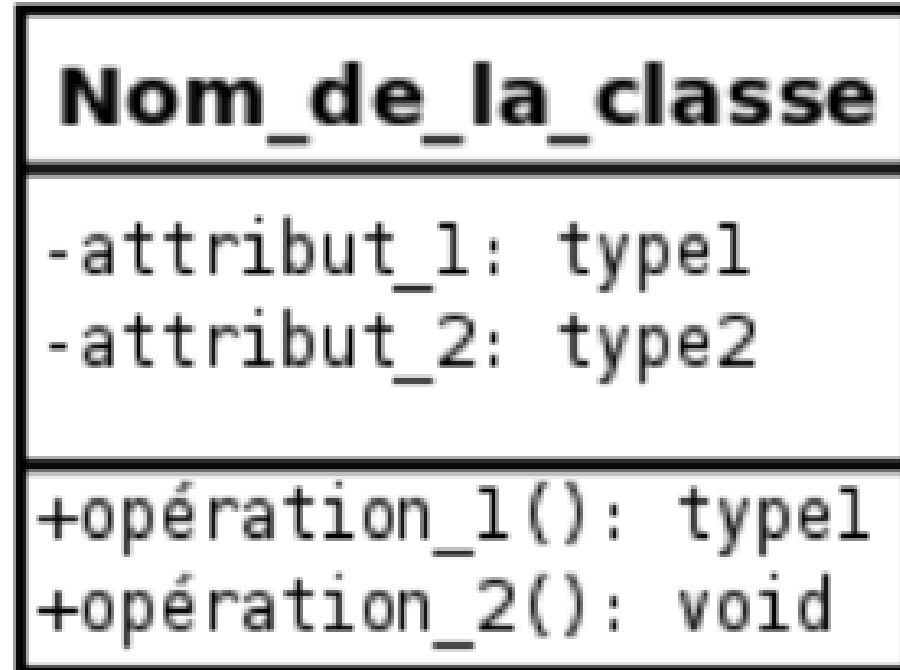
Exemple : Voiture

- Le 2ème compartiment contient les attributs : ce sont les données encapsulées dans les objets de cette classe.

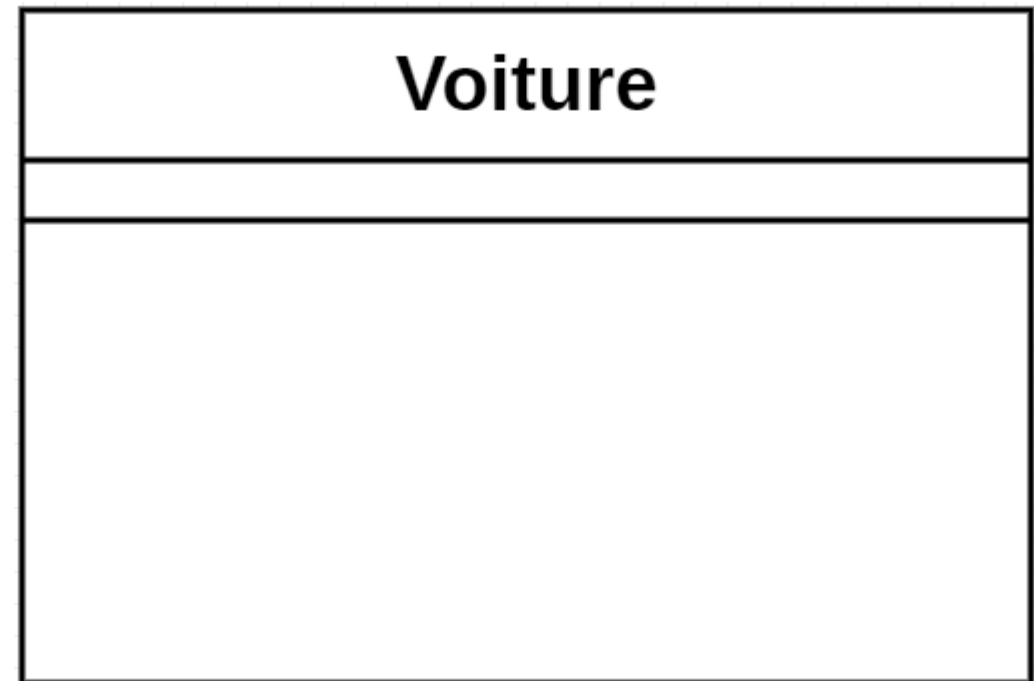
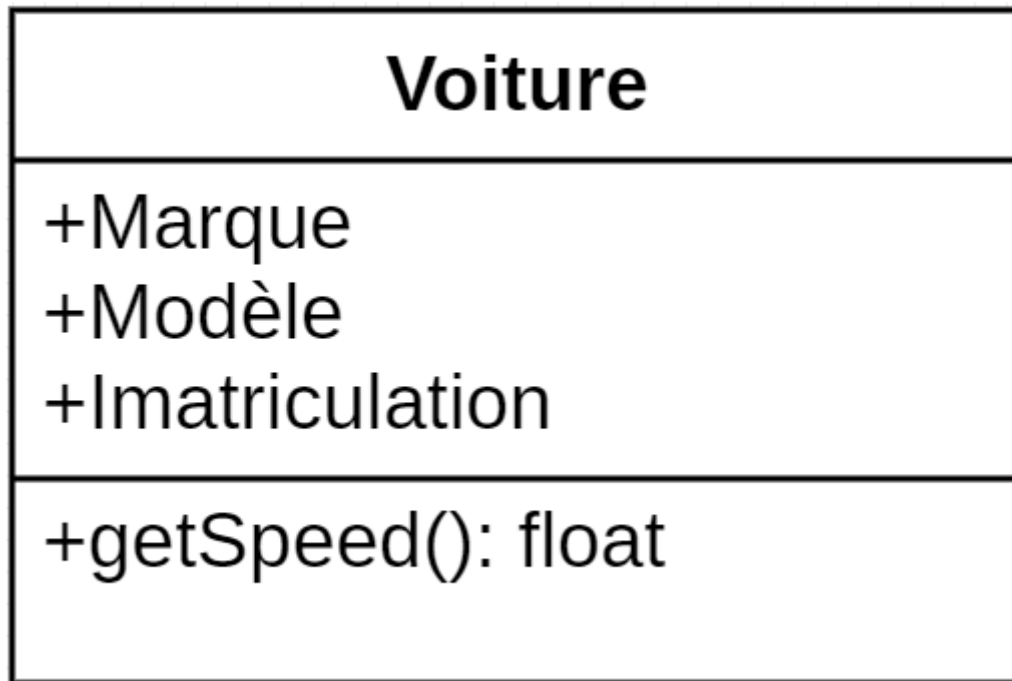
Exemple : vitesse courante, numéro d'immatriculation, etc.

Représentation graphique d'une classe avec UML

Le 3ème compartiment contient les opérations (méthodes) : les fonctionnalités/services assurées par les objets de la classe.



Représentation graphique d'une classe avec UML



Encapsulation et Visibilité



Encapsulation

L'encapsulation est un mécanisme consistant à **rassembler les données et les méthodes au sein d'une structure** (vue interne de l'objet) en **empêchant l'accès aux données par un autre moyen que les services proposés**.

Ces services accessibles aux utilisateurs de l'objet définissent ce que l'on appelle l'interface de l'objet (**vue externe de l'objet**).

L'encapsulation permet de **garantir l'intégrité des données** contenues dans l'objet.

L'encapsulation permet de définir des niveaux de visibilité des éléments d'un conteneur.

Visibilité

La visibilité déclare la possibilité pour un élément de modélisation de référencer un élément qui se trouve dans un espace de noms différent de celui de l'élément qui établit la référence.

Il existe quatre visibilités prédéfinies :

Visibilité

+ public

élément non encapsulé visible par tous.

- private

élément encapsulé visible seulement dans la classe.

protected

élément encapsulé visible dans la classe et dans les sous-classes.

~ package

élément encapsulé visible dans les classes du même paquetage

Visibilité des attributs

Voiture
-Marque -Modèle -Immatriculation -Speed
+getSpeed(): float +getMarque(): String

Dans la pratique, lorsque des attributs doivent être accessibles de l'extérieur, il est préférable que cet accès ne soit pas direct, mais se fasse par l'intermédiaire d'opérations (Accesseurs getter/setter)

Attributs et opérations



Attributs d'instance

Définir un attribut d'instance par :

- Un **nom** : doit être unique dans la classe
- Un **type** de données : (<type>) peut être un nom de classe, un nom d'interface ou un type de donnée prédéfini
- Une **visibilité**
- Une **valeur initiale** (<valeur_par_défaut>)
- La **multiplicité** ([multiplicité]) d'un attribut précise le nombre de valeurs que l'attribut peut contenir. On la définit entre []. Par défaut, la multiplicité est de 1.
- Une **contrainte** : lorsque la multiplicité est > 1, il est possible d'ajouter une contrainte ({<contrainte>}) pour préciser si les valeurs sont ordonnées ({ordered}) ou non ({list}).

Attributs d'instance

Syntaxe de déclaration d'un attribut :

```
<visibilité> [/] <nom_attribut> : <type> [  
  '[multiplicité]' [{contrainte}] ] [ = <valeur> ]
```

Attributs d'instance

Exemple :

L'attribut sous-titre est optionnel

UML indique ce caractère optionnel
en ajoutant une multiplicité [0..1]
derrière l'attribut

Livre
- titre : String - sous-titre : String [0..1] - ISBN : int - langue : String - date de parution : Date - Prix : float

Méthodes d'instance

Dans une classe, une opération doit être unique.

Quand le nom d'une opération apparaît plusieurs fois avec des paramètres différents, on dit que l'opération est surchargée.

Syntaxe de déclaration d'une méthode:

```
<visibilité> <nom> ([paramètre1], [paramètre2]...)  :  
[<type_renvoyé>] [{<propriétés>}]
```

Méthodes d'instance

Définir une méthode d'instance par:

- Un **nom** de méthode
- Les **paramètres** avec leur type
- Une **visibilité**
- Le **type** de la valeur de retour : peut être un nom de classe, un nom d'interface ou un type de données prédéfini
- Les **propriétés** (<propriétés>) correspondent à des contraintes ou à des informations complémentaires comme les exceptions, les pré-conditions, les post-conditions ou encore l'indication qu'une méthode est abstraite (abstract), etc.

Méthodes d'instance

Direction des paramètres des méthodes:

La syntaxe de définition d'un paramètre (<paramètre>) est la suivante :

```
[direction] <nom_paramètre>:<type> [ '['multiplicité']'  
] [= <valeur>]
```

Méthodes d'instance

La direction de transmission d'un paramètre peut être:

- **in** : paramètre d'entrée passé par valeur. Les modifications du paramètre ne sont pas disponibles pour l'appelant. C'est le comportement par défaut.
- **out** : paramètre de sortie uniquement. Il n'y a pas de valeur d'entrée et la valeur finale est disponible pour l'appelant.
- **inout** : paramètre d'entrée/sortie. La valeur finale est disponible pour l'appelant.

Exemple : la classe Horloge

La méthode `régler(h : int, m : int) : void`

L'appelant de la méthode veut affecter les attributs **h** et **m** avec des valeurs qu'il va donner.

La direction des paramètres sera alors **in** (par défaut) : `régler(in h : int, in m : int) : void`

Horloge
-heures: int -minutes: int
+régler(h : int, m : int): void +afficher(): void +getTime(out h: int, out m: int): void

Valeurs par défauts des attributs et des paramètres des méthodes

On indique les valeurs par défaut des attributs lors de leur construction et les valeurs par défaut des paramètres des méthodes s'ils ne sont pas clairement spécifiés lors de l'appel.

Horloge
-heures: int = 0 -minutes: int = 0
+régler(h : int = 0, m : int = 0): void +afficher(): void +getTime(out h: int, out m: int): void

Attributs et méthodes statiques

- Une classe peut contenir des attributs et des méthodes qui lui sont propres et auxquels on peut accéder sans nécessairement instancier des objets.
- Un attribut de classe n'appartient pas à un objet en particulier mais à toute la classe (il n'est pas instancié avec l'objet).
- Il garde une valeur unique et partagée par toutes les instances de la classe.
- Une méthode de classe ne peut manipuler que des attributs statiques et ses propres paramètres

Attributs et méthodes statiques

Graphiquement, un attribut ou une méthode de classe est représenté par un nom souligné.

Gasoil
<u>-quantité: double {quantité < 100}</u> <u>-prixParLitre: double = 16</u>
+setQuantité(quantité: double): void +getQuantité(): double <u>+fixePrix(parLitre: double): void</u>

Contraintes



Contraintes

Une contrainte est une condition écrite entre 2 accolades {} elle peut être exprimée dans :

- Un langage naturel (description textuelle)
- Un langage formel (C++, java, ...).

Les contraintes sont appliquées sur les attributs ou les méthodes

Gasoil
-quantité: double {quantité < 100} <u>-prixParLitre: double = 16</u>

Contraintes

{readOnly} : la valeur de l'attribut ne peut plus être modifiée une fois initialisée

{ordered} : pour les attributs de type collection. C'est pour préciser si les valeurs sont ordonnées

{list} : similaire à {ordered} mais pour préciser si les valeurs ne sont pas ordonnées (par défaut)

{unique} : aucun doublon dans les valeurs de la collection

{not null} : L'attribut doit à tout prix être initialisé

{query} : appliqué sur une méthode si son code ne modifie nullement l'état de l'objet (aucun de ses attributs)

Attributs calculés (dérivés)



Attributs dérivés

- Un attribut dérivé est un attribut dont la valeur peut être calculée à partir d'autres informations disponibles dans le modèle, par exemple d'autres attributs de la même classe, ou de classes en association.
- Cet attribut, qui pourrait donc être considéré comme redondant, est néanmoins gardé par l'analyste s'il correspond à un concept important aux yeux de l'expert métier.
- Il est noté en UML avec un « / » avant son nom et suivi d'une contrainte permettant de le calculer.

Attribut dérivé

Gasoil

-quantité: double {quantité < 100}
-prixParLitre: double = 16
-/prixDuPlein: double {prixDuPlein = quantité * prixParLitre}

+setQuantité(quantité: double): void
+getQuantité(): double {query}
+fixePrix(parLitre: double): void

Multiplicité (cardinalité)



Multiplicité

- La multiplicité indique le nombre de valeur que l'attribut peut contenir. L'attribut est souvent un tableau de valeurs statique ou dynamique (collection).
- La multiplicité se note entre crochets [] après le type de valeur que contient l'attribut.
- On peut préciser la multiplicité également pour un paramètre de méthode au niveau de la signature

Multiplicité

Une station météo doit relever la température à intervalle de temps régulier.

Elle doit pouvoir stocker 100 relevés.

RelevéMétéo
-Température: int[100]

Cardinalité	Signification
0..1	Zéro ou une fois
1..1	Une et une seule fois
0..* (ou *)	De zéro à plusieurs fois
1..*	De une à plusieurs fois
m..n	Entre m et n fois
n..n (ou n)	n fois

Constructeurs et destructeurs



Constructeurs et destructeurs

Les stéréotypes peuvent être utilisés pour identifier des opérations particulières comme :

- les constructeurs : stéréotype « create »
- et le destructeur : stéréotype « destroy ».

Dans UML, les constructeurs et les destructeurs sont des méthodes spéciales qui ont le même nom que la classe (Formalisme Java)

Constructeurs et destructeurs

Un stéréotype est un mot-clé inventé par les utilisateurs.

Il existe des stéréotypes définis comme <<include>>, <<create>>, etc.

Mais on peut créer notre propre mot-clé

Horloge
-heures: int = 0 -minutes: int = 0
«create»+Horloge(h: int, m: int): void +régler(h : int = 0, m : int = 0): void +afficher(): void +getTime(out h: int, out m: int): void «destroy»+Horloge(): void

Modèles de classe



Modèles de classe

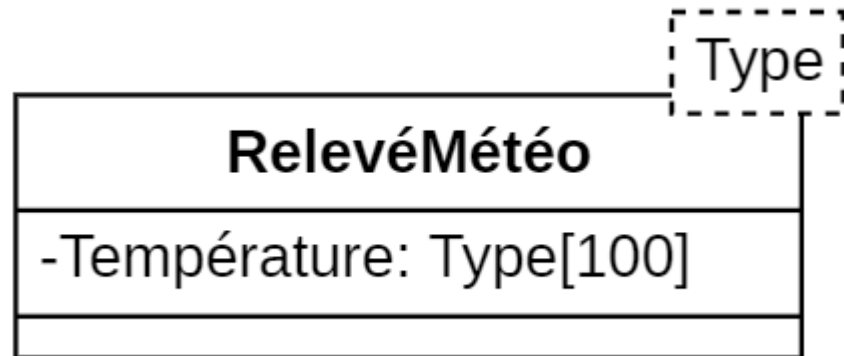
Les modèles (templates) sont une fonctionnalité avancée de l'orientée objet.

Un modèle est une classe paramétrée qui permet ainsi de choisir le type des attributs

Ces modèles de classes sont particulièrement utiles pour toutes les collections qui stockent des valeurs d'un même type, soit sous forme de tableaux dynamiques ou de listes.

Modèles de classe

Le paramètre est précisé dans le coin supérieur droit dans un rectangle dont les côtés sont en pointillés.



Relations entre classes



Relation de dépendance

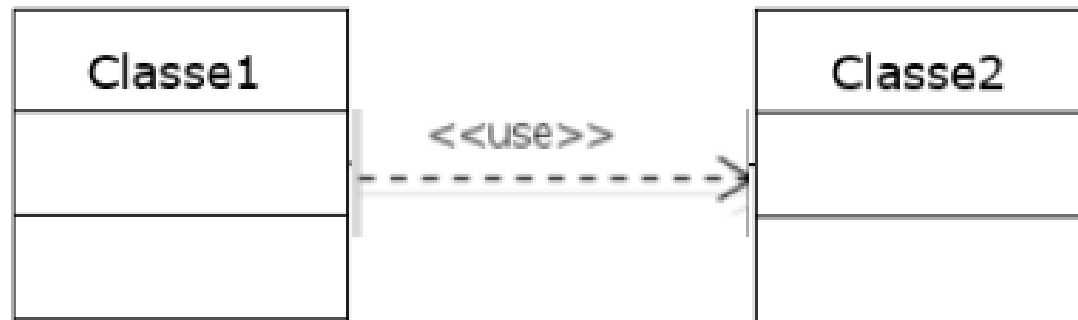
La dépendance est la forme la plus faible de relation entre classes.

Une dépendance entre deux classes autorise simplement une classe à utiliser des objets d'une autre classe

Il s'agit d'une relation transitoire, au sens où la première interagit brièvement avec la seconde sans conserver à terme de relation avec elle (liaison ponctuelle).

Relation de dépendance

La dépendance Notation UML : représentée par un trait discontinu orienté, reliant les deux classes. La dépendance est souvent stéréotypée par <<use>> (<<utilise>>)



Relation de dépendance

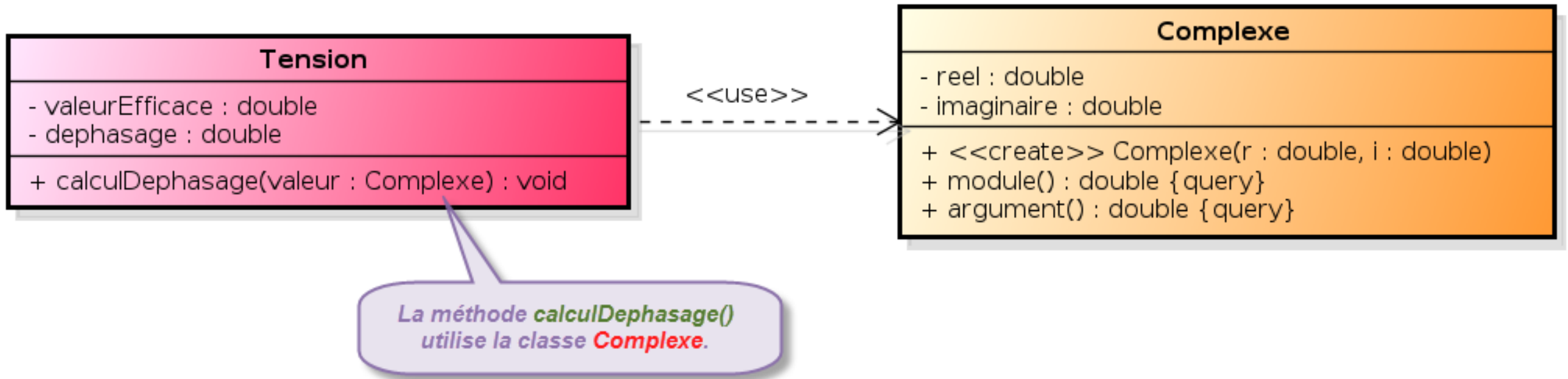
Une relation de dépendance est habituellement utilisée si :

- Une classe utilise un objet d'une autre classe comme argument dans la signature d'une méthode
- Un objet de l'autre classe est créé à l'intérieur de la méthode.

Dans les deux cas la durée de vie de l'objet est très courte, elle correspond à la durée d'exécution de la méthode

Relation de dépendance

Exemple :



Associations entre classes

Par contre, l'association est une relation plus forte. Elle indique qu'une classe est en relation avec une autre pendant un certain laps de temps.

La ligne de vie des deux objets concernés ne sont cependant pas associés étroitement (un objet peut être détruit sans que l'autre le soit nécessairement).

Associations entre classes

Notation UML : L'association est représentée par un simple trait continu, reliant les deux classes. Le fait que deux instances soient ainsi liées permet la navigation d'une instance vers l'autre, et vice versa (chaque classe possède un attribut qui fait référence à l'autre classe).



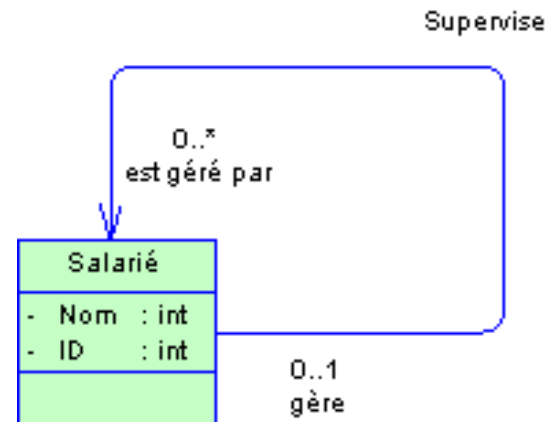
Associations entre classes

Il existe plusieurs types d'associations :

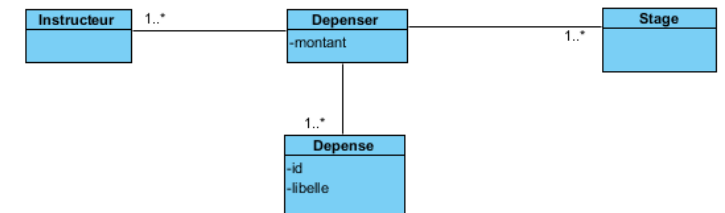
Association binaire
entre 2 classes



Association réflexive
entre la classe et elle-même



Association n-aire
entre n classes ($n \geq 3$)



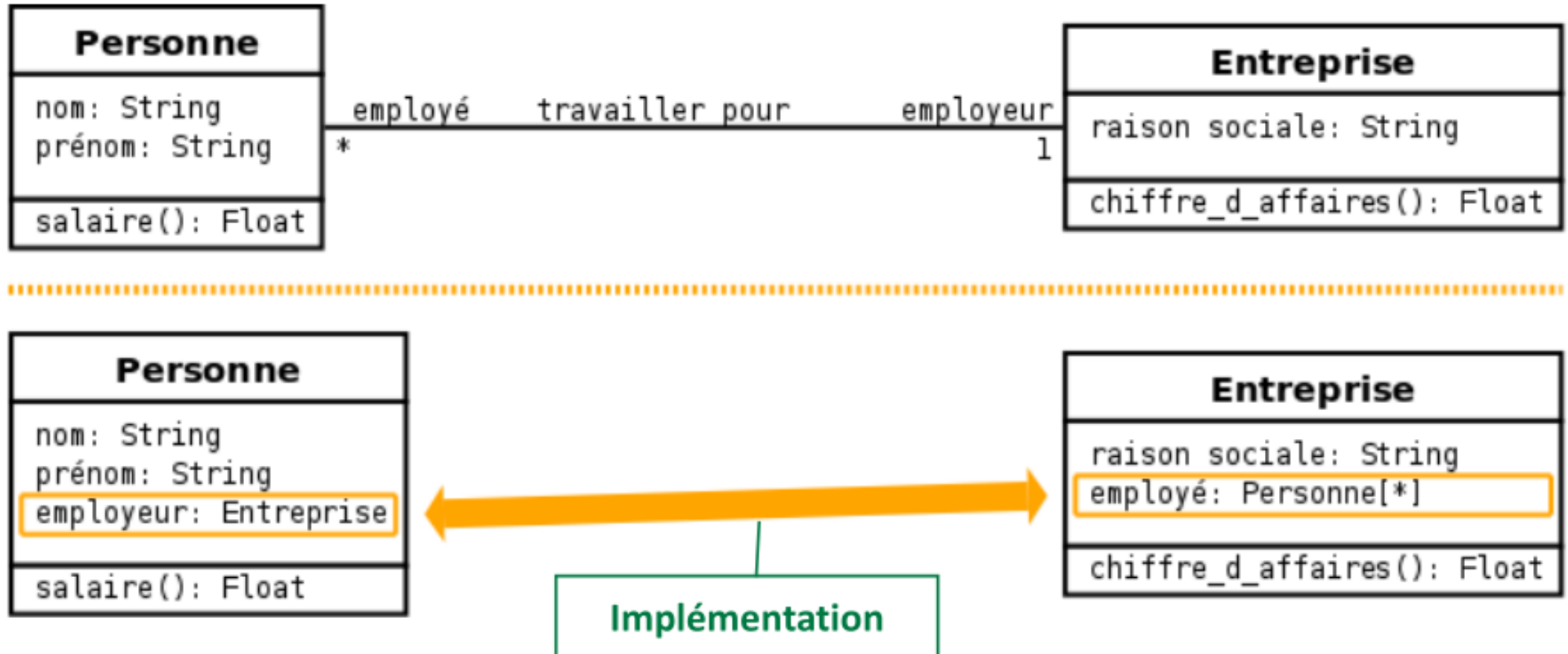
Associations binaire - Implémentation

Une association binaire est matérialisée par un trait plein entre les classes.

Si on veut implémenter l'association dans un langage orienté objet, chaque classe de l'association contiendra une référence (ou un pointeur) de l'objet de la classe associée sous la forme d'un attribut.

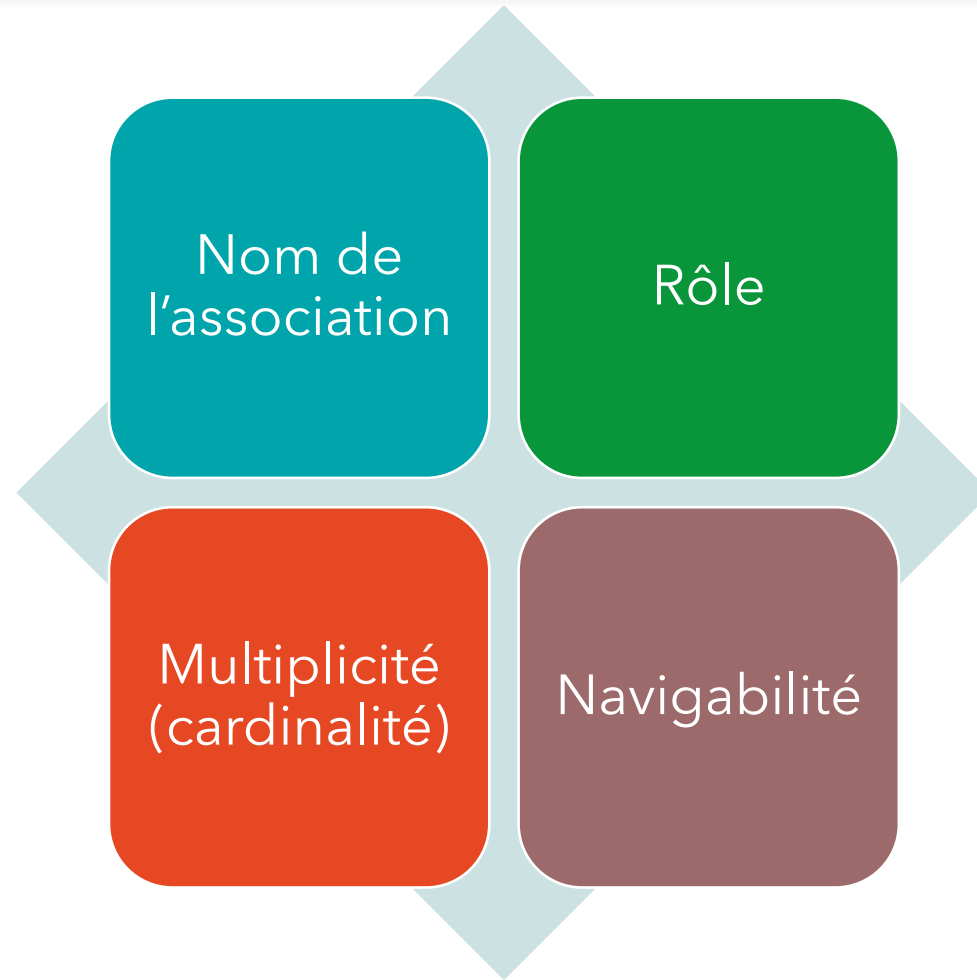
Associations binaire - Implémentation

Exemple



Associations entre classes

Nous pouvons détailler l'association en indiquant :



Nom de l'association

L'association peut être ornée d'un texte, avec un éventuel sens de lecture, qui permet de nous informer de l'intérêt de cette relation. Ce nom n'est absolument pas exploité dans le code.



Rôle des extrémités de l'association

Chaque extrémité d'une association peut être nommée.

Ce nom est appelé rôle et indique la manière dont l'objet est vu de l'autre côté de l'association.



Multiplicité de l'association

La multiplicité (cardinalité) indique le nombre d'instances de classe étant en relation avec la classe située à l'autre extrémité de l'association.

Par défaut, la cardinalité est égale à 1.



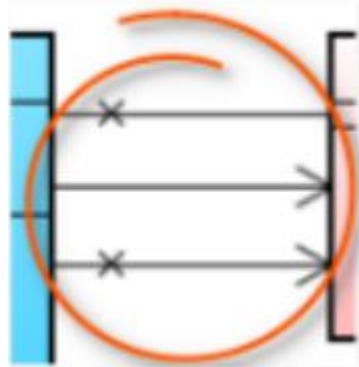
Navigabilité

Navigabilité bidirectionnelle

- par défaut
- chaque classe possède un attribut qui fait référence à l'autre classe en association.
- plus complexe à réaliser
- A éviter dans la mesure du possible

Navigabilité unidirectionnelle

- une seule classe possède un attribut qui fait référence à l'autre classe plus fréquente
- la première classe peut solliciter une deuxième et que l'inverse est impossible
- peut se représenter de 3 façons différentes :
 - ✓ Une croix du côté de l'objet qui ne peut pas être sollicité
 - ✓ Une flèche du côté de l'objet qui peut être sollicité
 - ✓ Les 2 représentations précédentes à la fois



Navigabilité

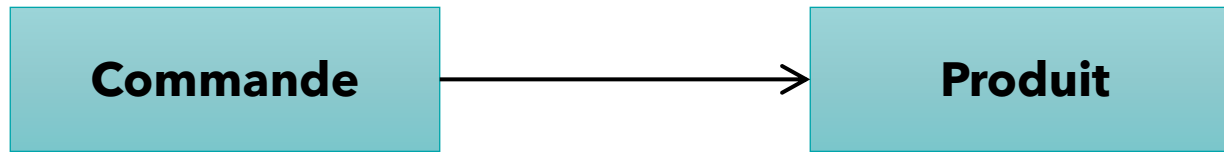
Exemple :

L'extrémité du côté de la classe Commande n'est pas navigable : les instances de la classe Produit ne stockent pas de liste d'objets du type Commande.

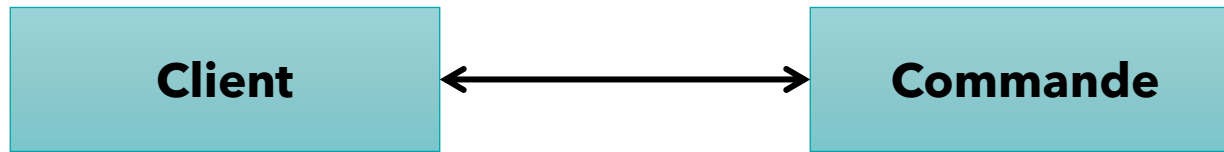


Navigabilité

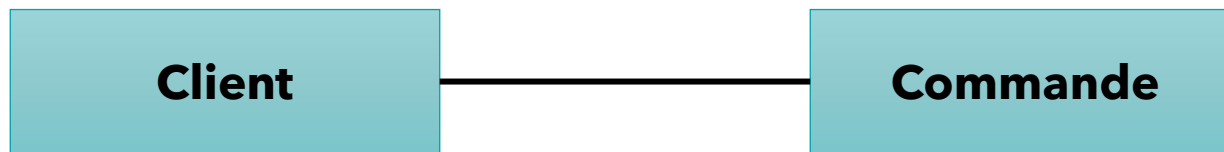
Exemple :



navigabilité unidirectionnelle: Le bon de commande peut voir le produit, mais le produit ne peut pas voir le bon de commande



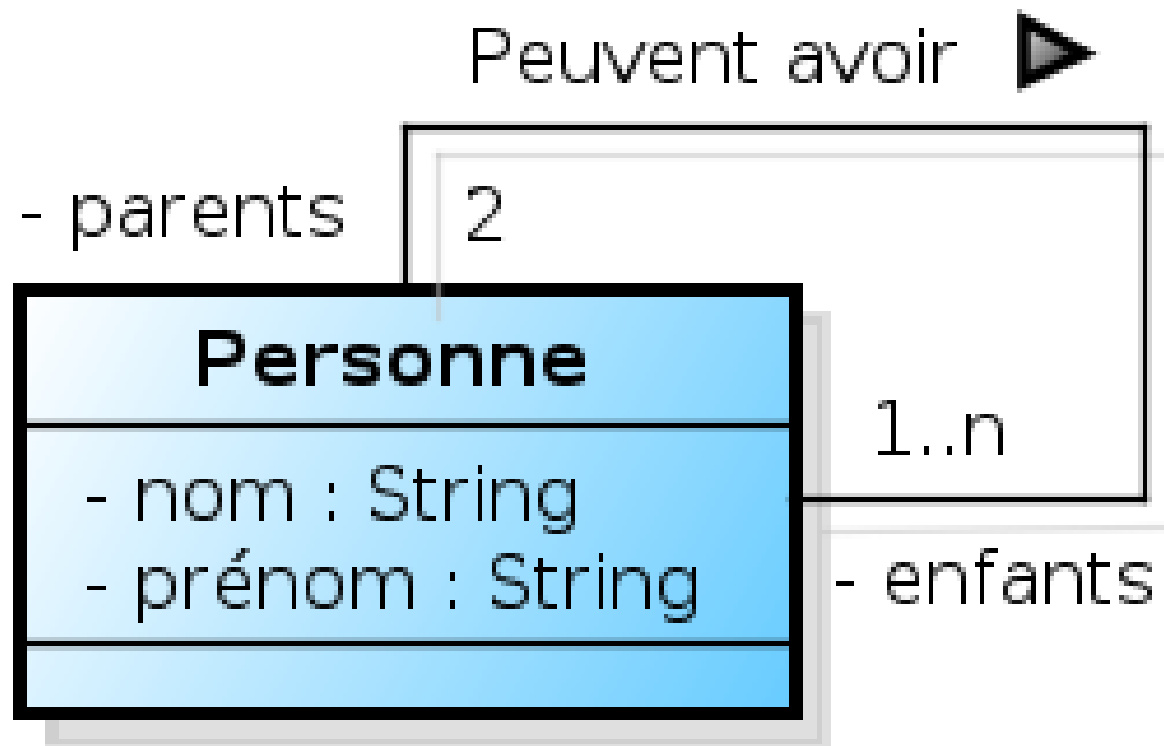
les flèches aux deux extrémités, ainsi que l'absence de flèches, indiquent la même chose.



navigabilité bidirectionnelle: le client peut voir ses bons de commande et un bon de commande peut voir le client respectif

Association réflexive

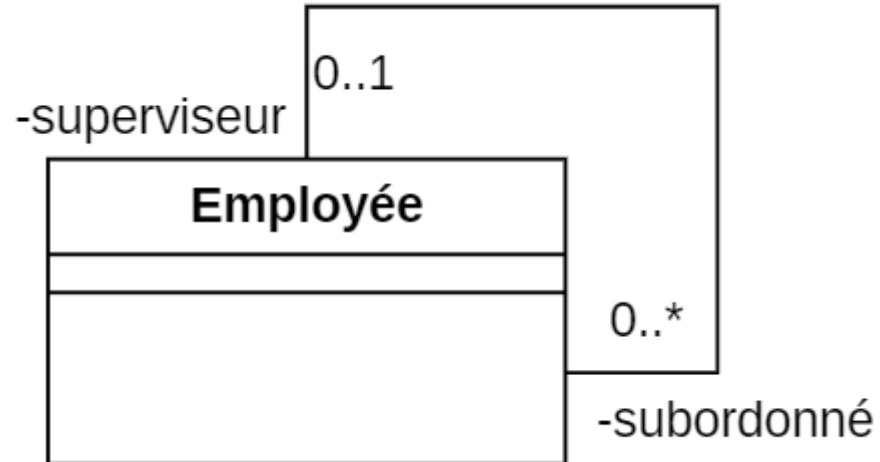
Une association réflexive (ou récursive) : lie une classe avec elle-même



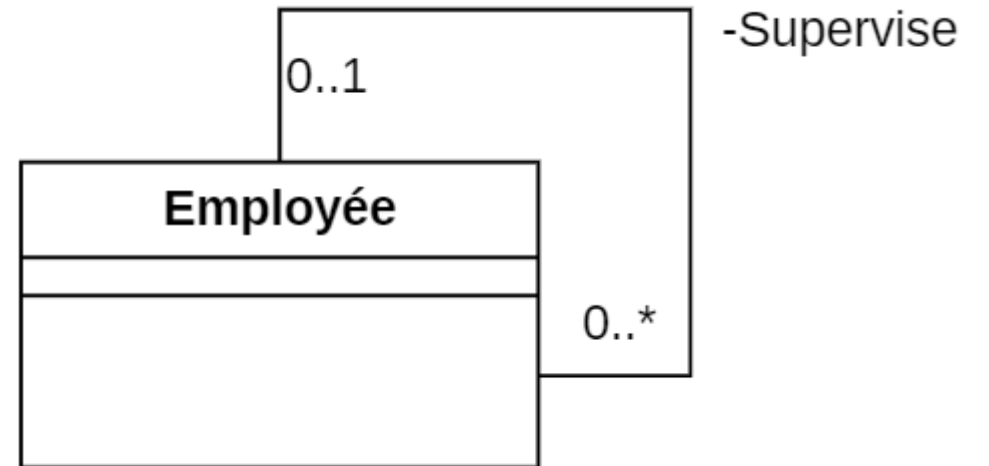
Association réflexive

Exemple:

Utilisant les rôles



Utilisant le nom de l'association

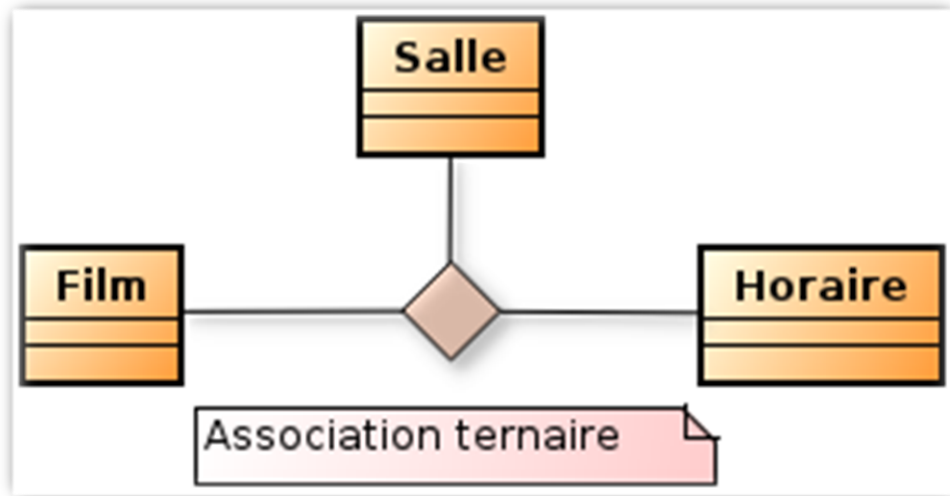


Association n-aire

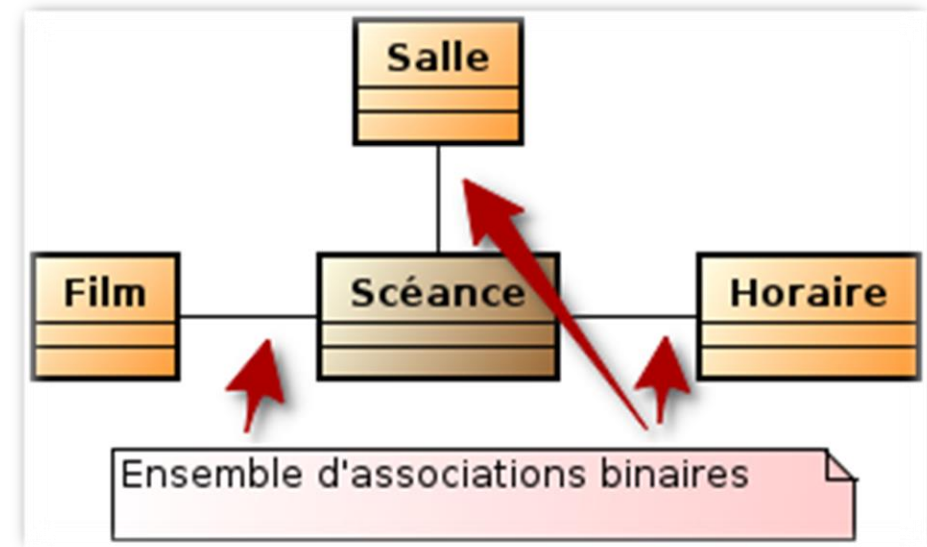
- L'association n-aire : est une association qui lie plus de 2 classes entre elles, est une. ($n \geq 3$)
- Notation : L'association n-aire se représente par un losange d'où part un trait allant à chaque classe.
- L'association n-aire est imprécise, difficile à interpréter et souvent source d'erreur, elle est donc très peu utilisée.
- La plupart du temps, on se sert pour la modélisation au début du projet, puis elle est vite remplacée par un ensemble d'associations binaires afin de lever toute ambiguïté.

Association n-aire

Exemple 1 : Une séance de cinéma peut correspondre à l'association ternaire de 3 classes.

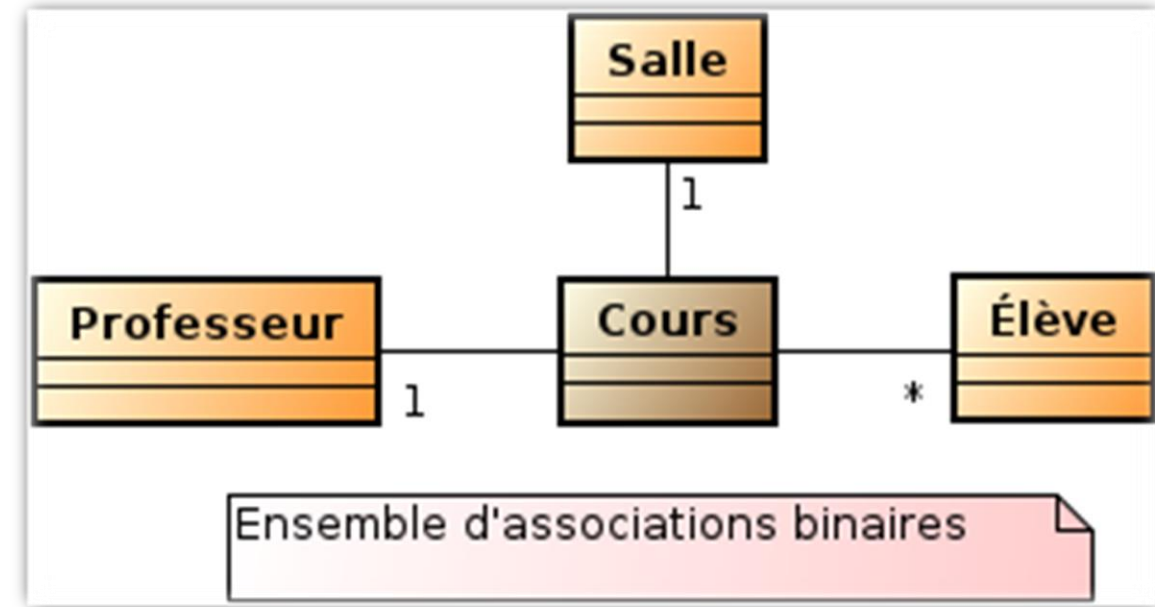
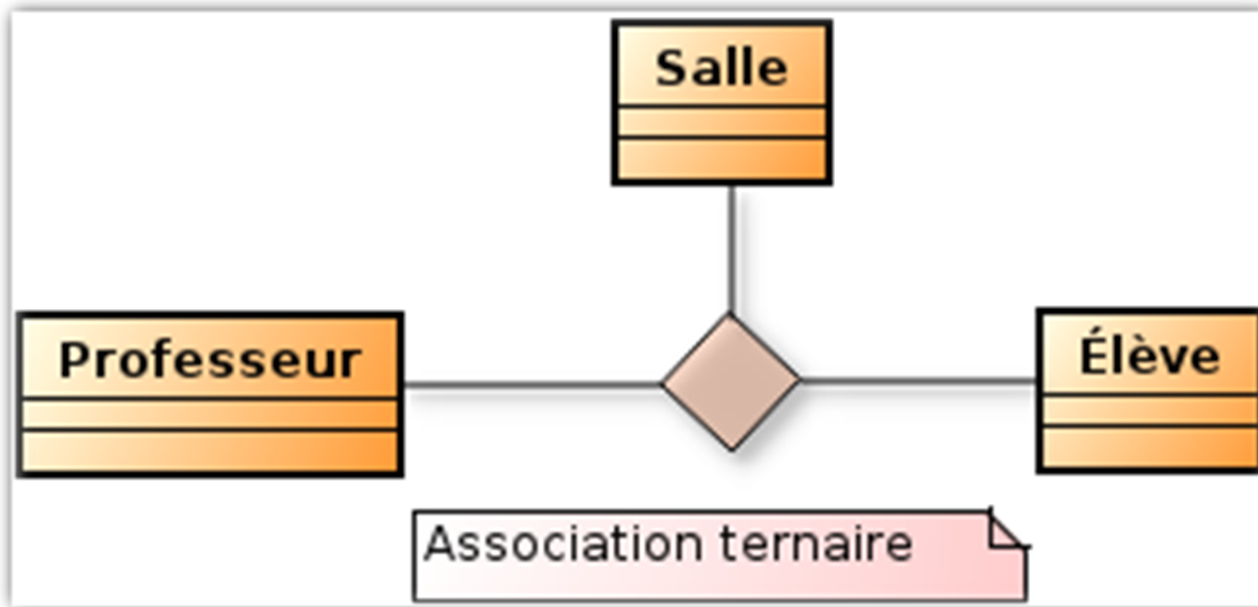


À remplacer par :



Association n-aire

Exemple 2 : Un cours peut correspondre à l'association ternaire de 3 classes (Salle, Professeur et Elève).



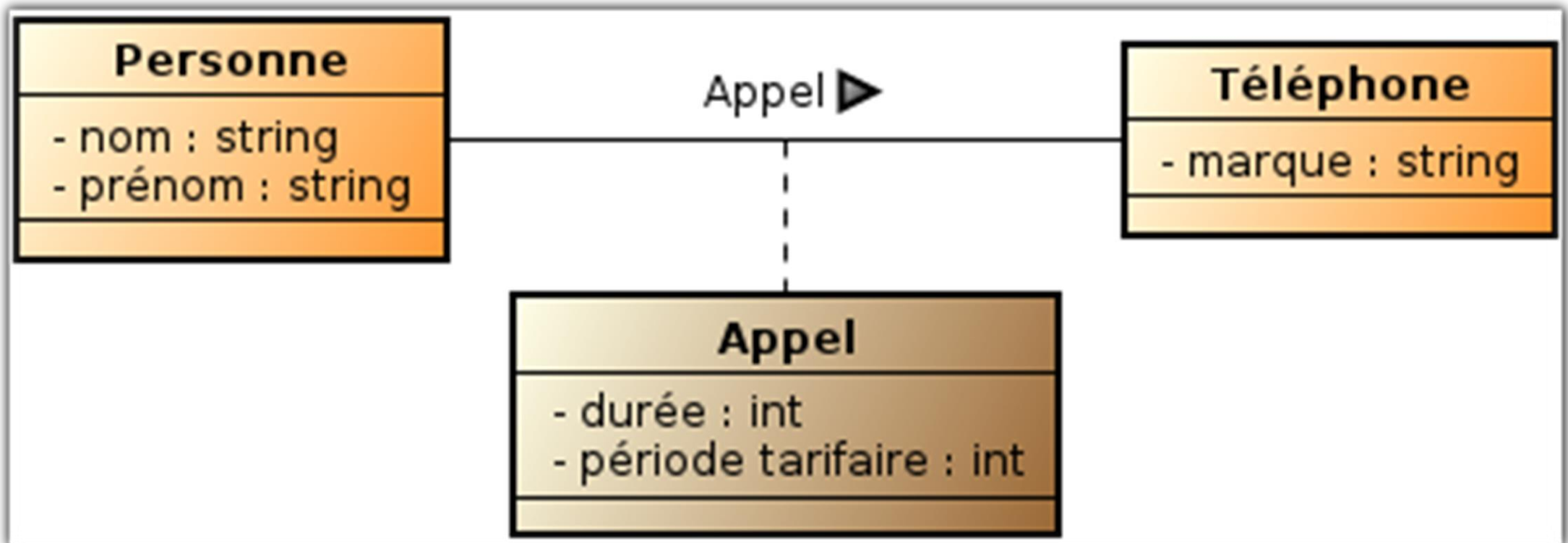
À remplacer par :

Classe association

- Une association peut apporter de nouvelles informations (attributs et méthodes) qui n'appartiennent à aucune des deux classes qu'elle relie et qui sont spécifiques à l'association.
- Ces nouvelles informations peuvent être représentées par une nouvelle classe attachée à l'association via un trait en pointillés.

Associations reliant plusieurs classes

Exemple:



Classe association

Exemple:

Lorsqu'une personne utilise un téléphone, il faut pouvoir mesurer la durée de l'appel et savoir à quel moment il a lieu afin de le tarifier.

On va ajouter donc deux attributs durée et période tarifaire qui n'appartiennent ni à la classe Personne ni à la classe Téléphone.

Ces deux attributs sont mis dans une nouvelle classe (la classe Appel) qui est attachée à l'association.

Classe association

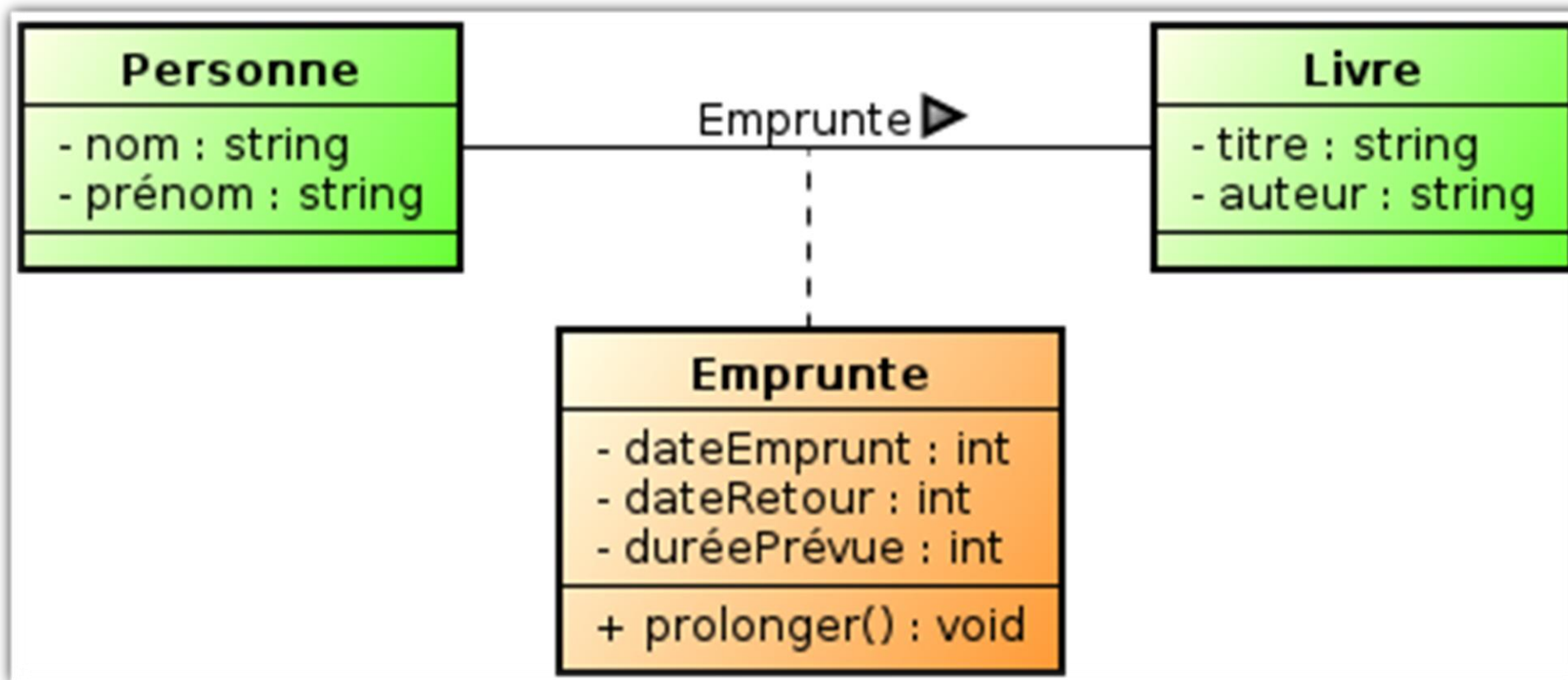
Remarque:

Comme pour l'association ternaire, nous pouvons convertir la classe association en un ensemble d'associations binaires.



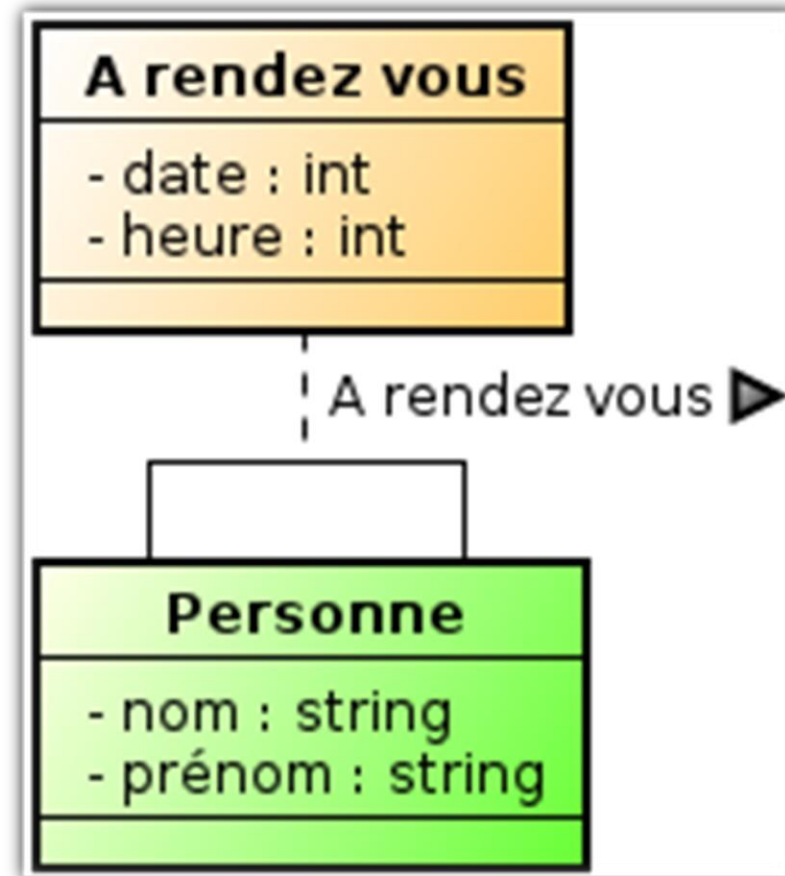
Classe association

Autres exemples:



Classe association

Autres exemples:



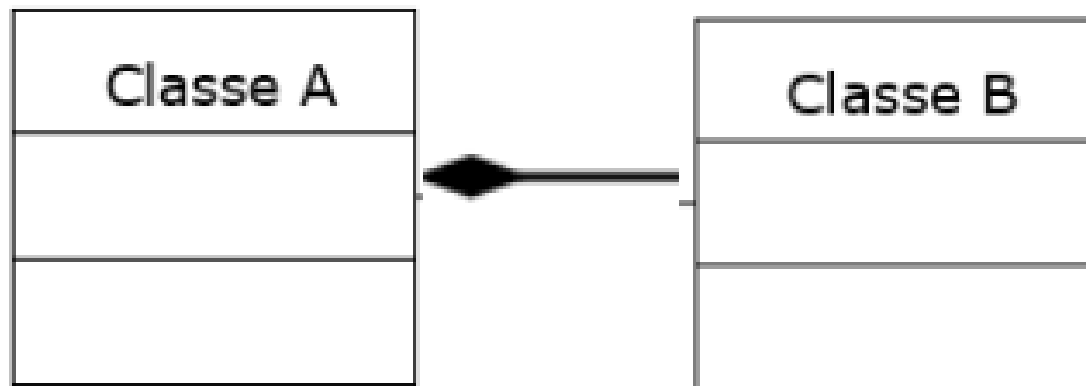
Associations particulières - Composition

La composition indique qu'un objet A (appelé conteneur) est constitué d'un autre objet B.

Cet objet A n'appartient qu'à l'objet B et ne peut pas être partagé avec un autre objet.

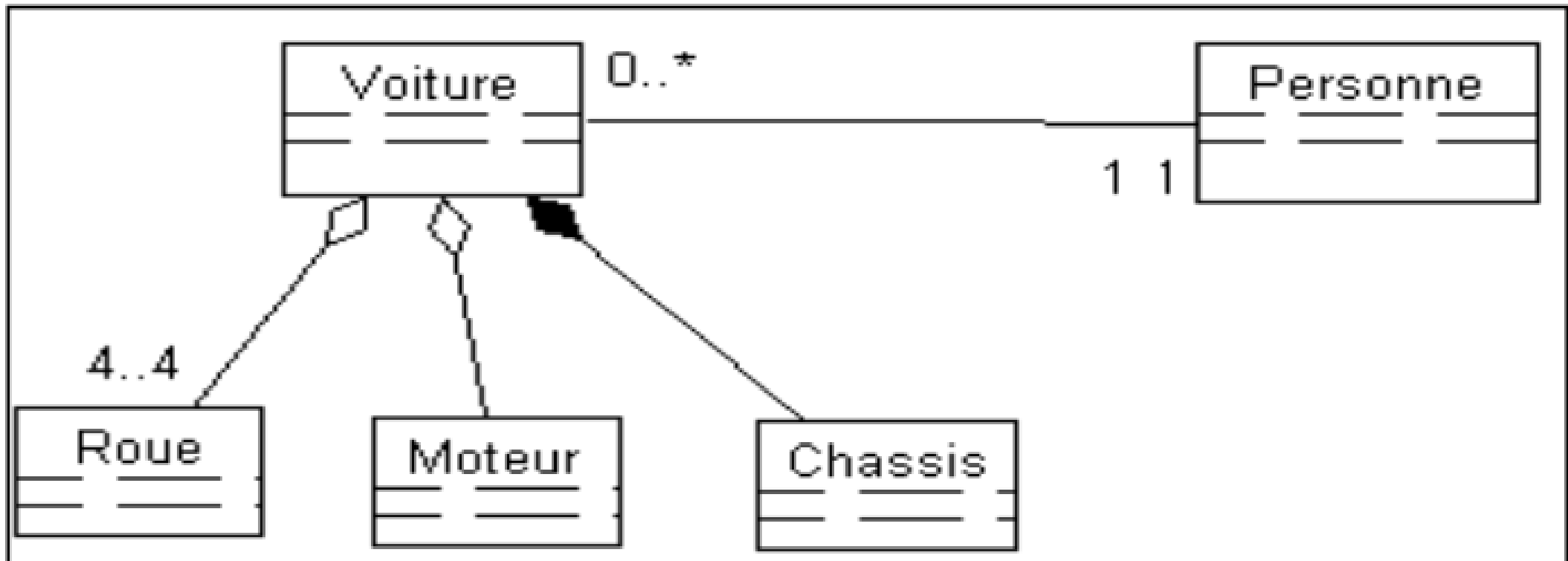
C'est une relation très forte, si l'objet A disparaît, alors l'objet B disparaît aussi.

Elle se représente par un losange plein du côté de l'objet conteneur



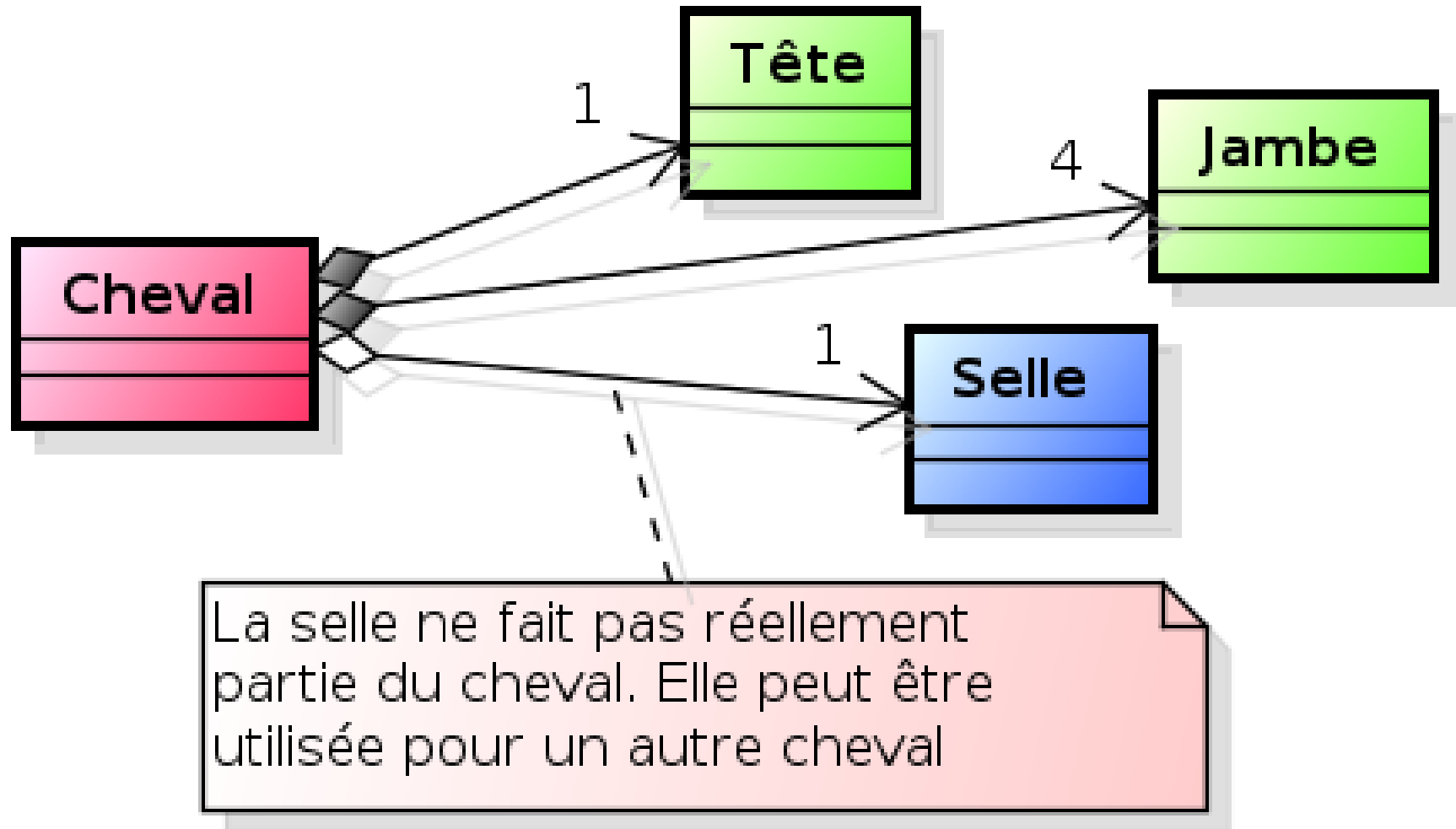
Associations particulières - Composition

Exemple : Une voiture est composée d'un châssis. Il ne peut pas être utilisé pour une autre voiture



Associations particulières - Composition

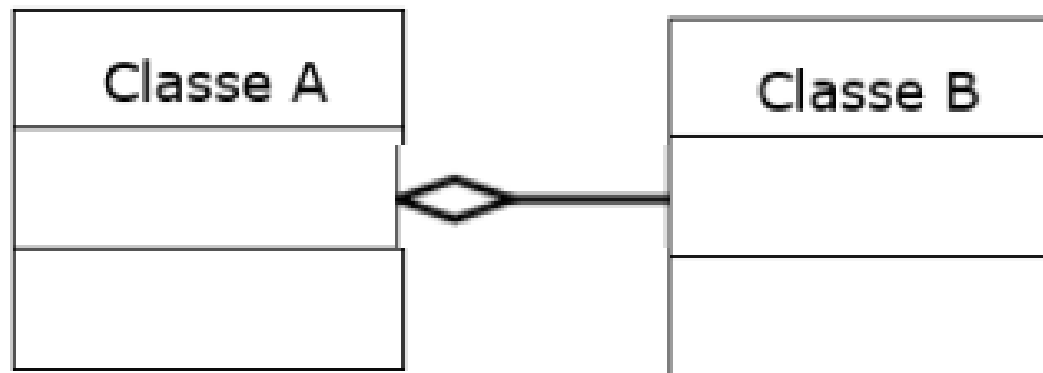
Exemple :



Associations particulières - Agrégation

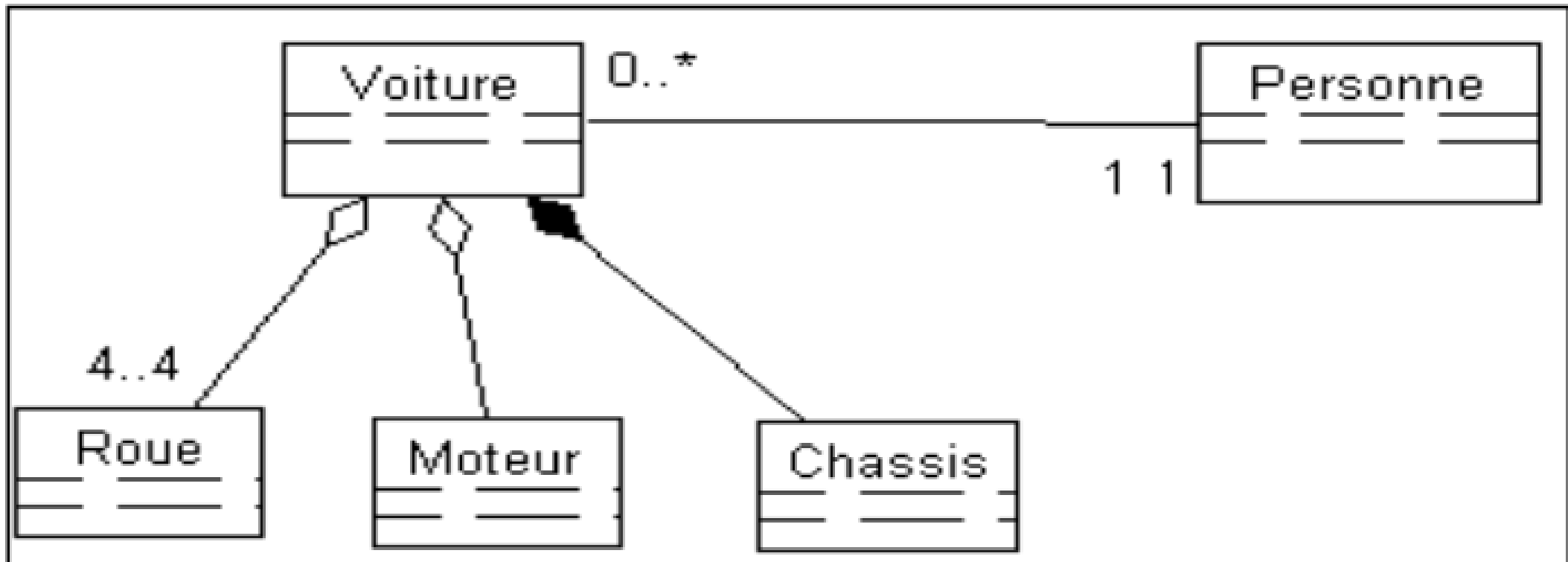
L'agrégation indique qu'un objet A possède un autre objet B, mais contrairement à la composition, l'objet B peut exister indépendamment de l'objet A.

La suppression de l'objet A n'entraîne pas la suppression de l'objet B. Elle se représente par un losange vide du côté de l'objet conteneur



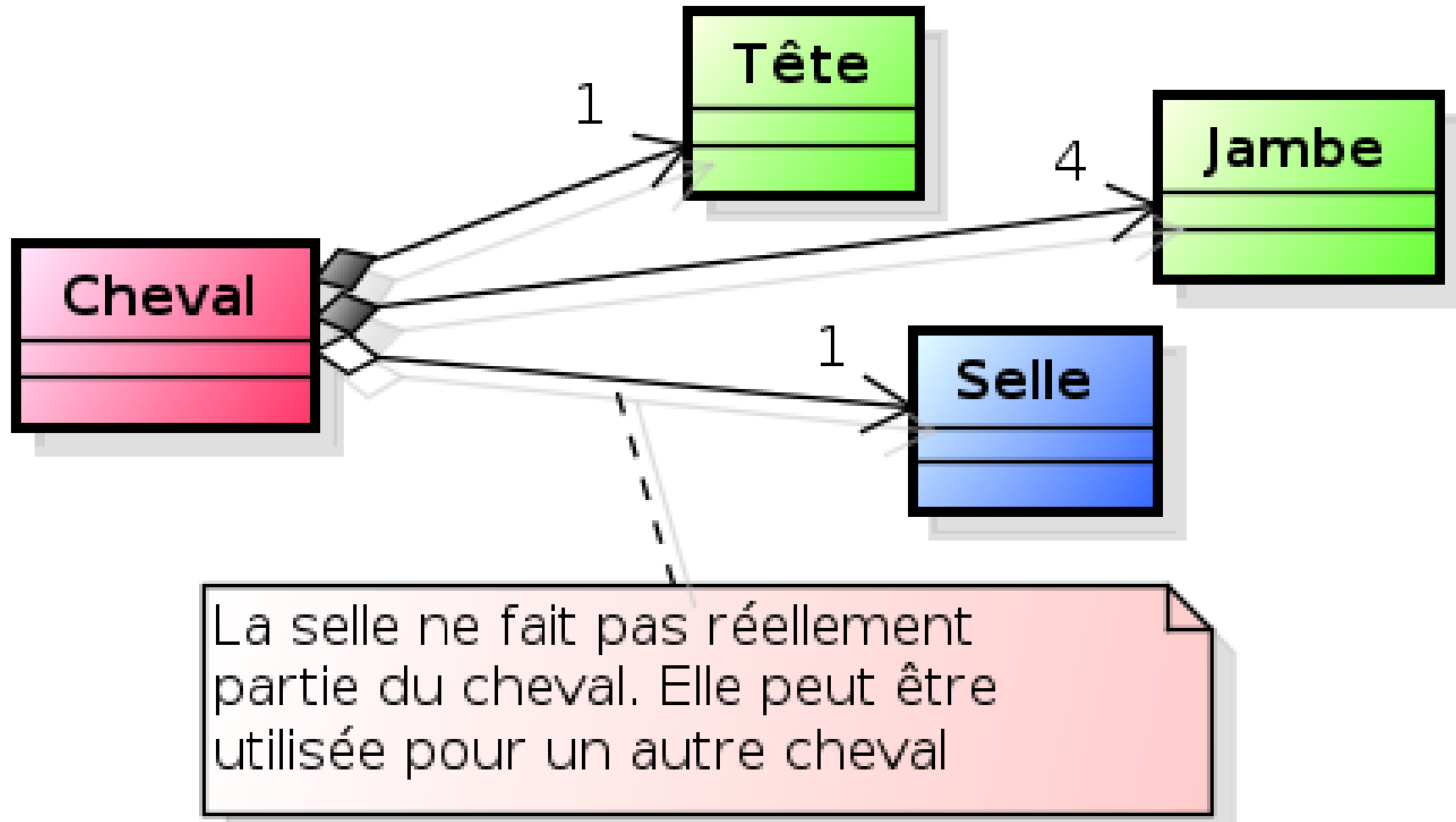
Associations particulières - Agrégation

Exemple : Une voiture est composée de 4 roues et un moteur. Ils peuvent être utilisés pour une autre voiture



Associations particulières - Agrégation

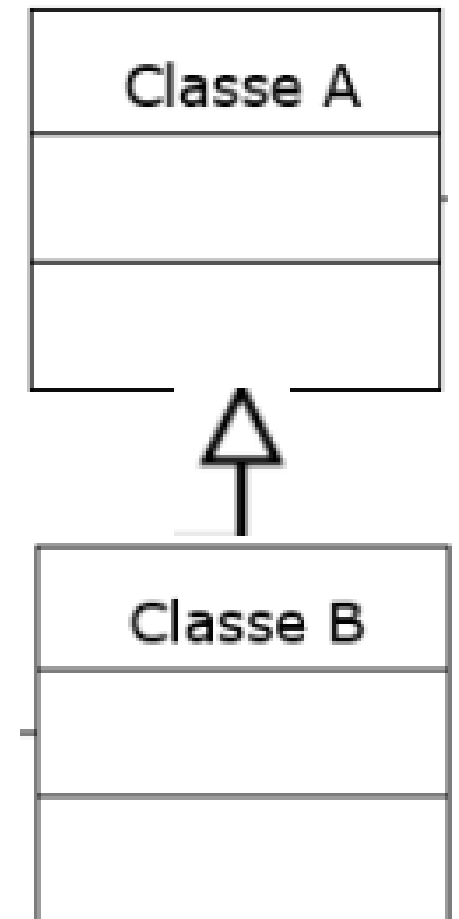
Exemple :



Relation d'héritage

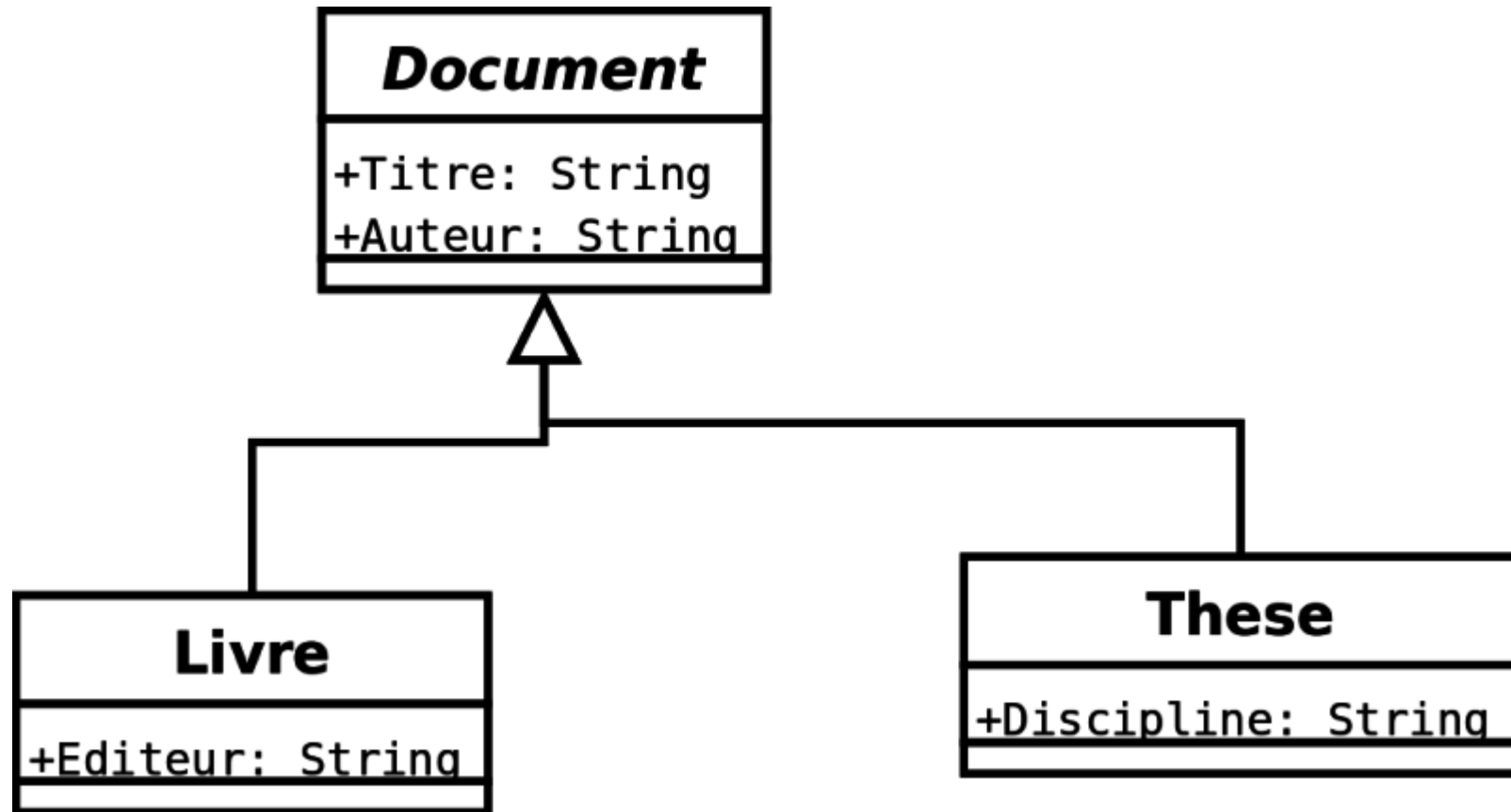
L'héritage indique qu'une classe B est une spécialisation d'une classe A. La classe B (appelé classe fille, classe dérivée ou sous classe) hérite des attributs et des méthodes de la classe A (appelée classe mère, classe de base ou super classe).

Notation : Il se représente par un triangle vide afin d'indiquer le sens de la généralisation (inverse de la spécialisation).



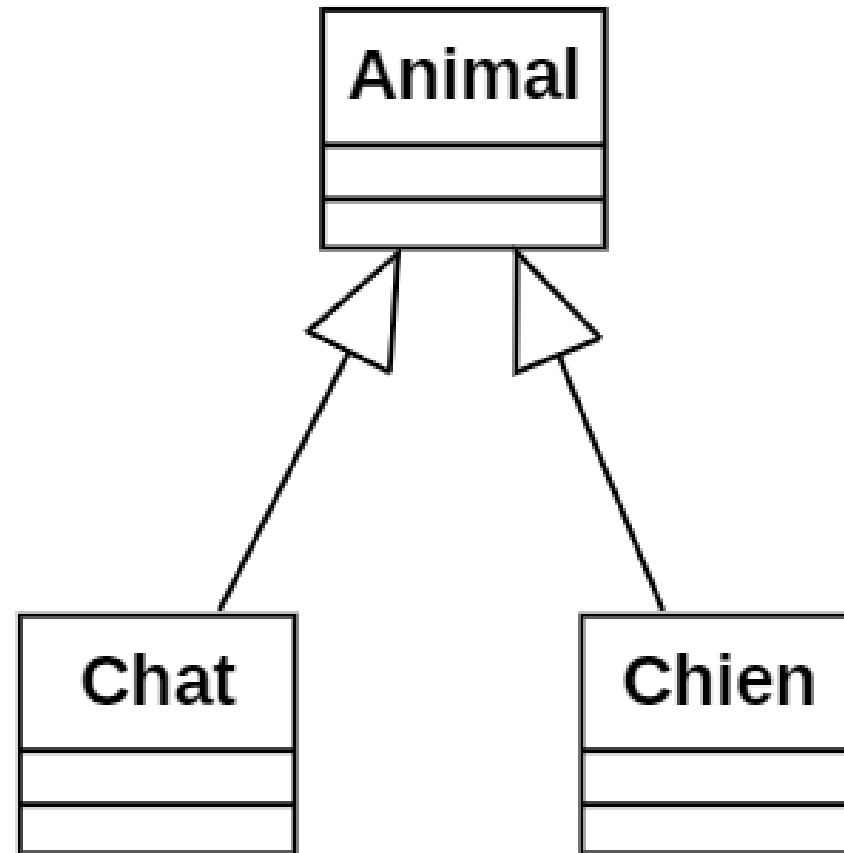
Relation d'héritage

Exemple :



Relation d'héritage

Exemple :



Classes concrètes et abstraites

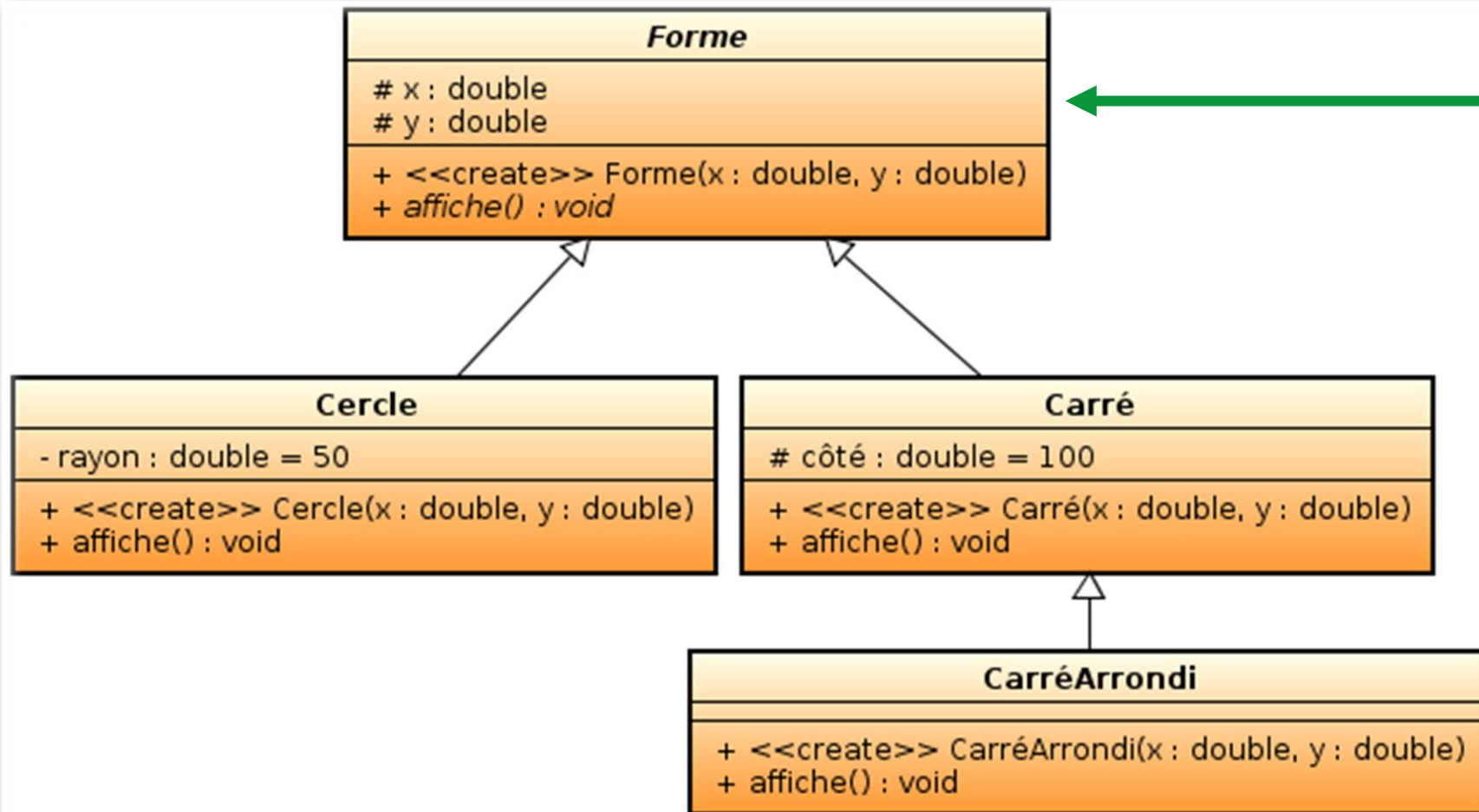


Classes concrètes et abstraites

En UML, une classe ou une méthode abstraite sont représentées avec une mise en italique du nom de la classe ou de la méthode.

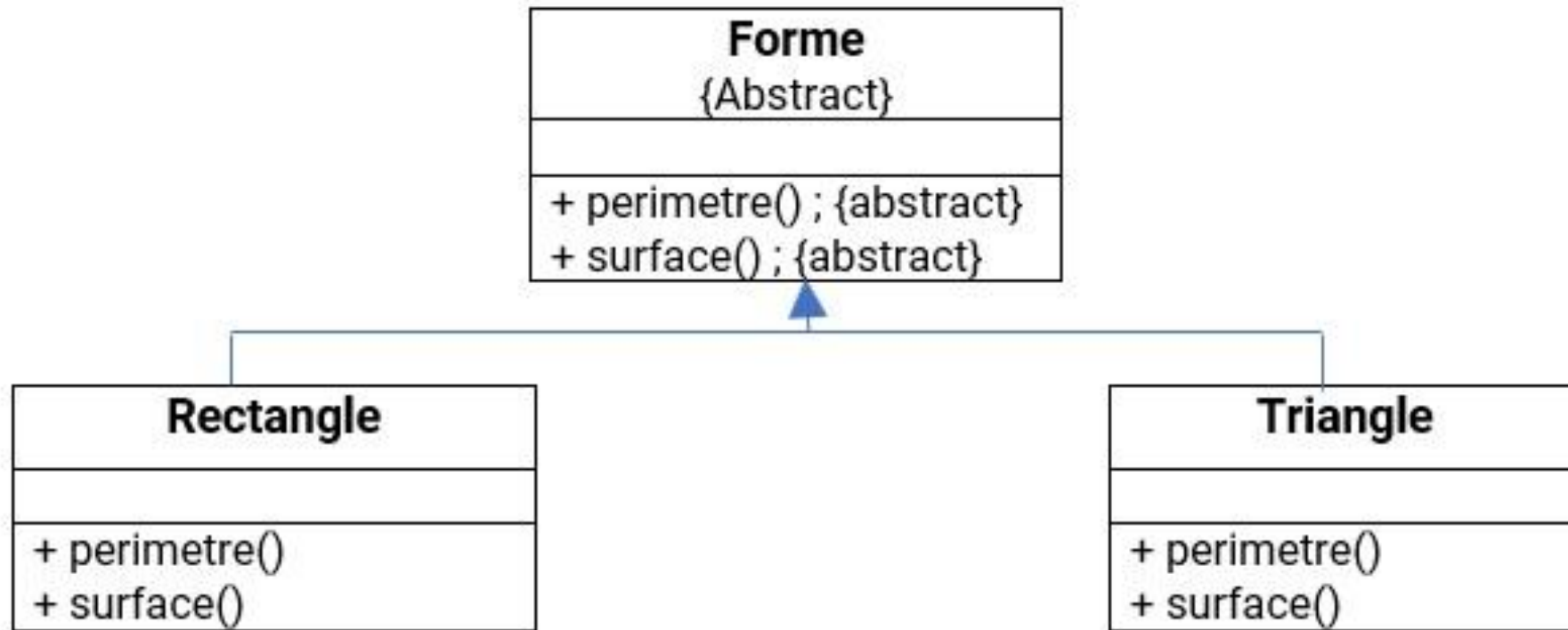
On peut aussi mettre le stéréotype <<abstract>>

Classes concrètes et abstraites



Classe abstraite

Classes concrètes et abstraites



Relation de réalisation et notion d'interfaces



Notion d'interfaces

L'interface est une classe particulière qui ne définit que des éléments d'interface.

Il peut s'agir de l'interface complète d'une entité, ou très généralement d'une partie d'interface qui sera commune à plusieurs entités.

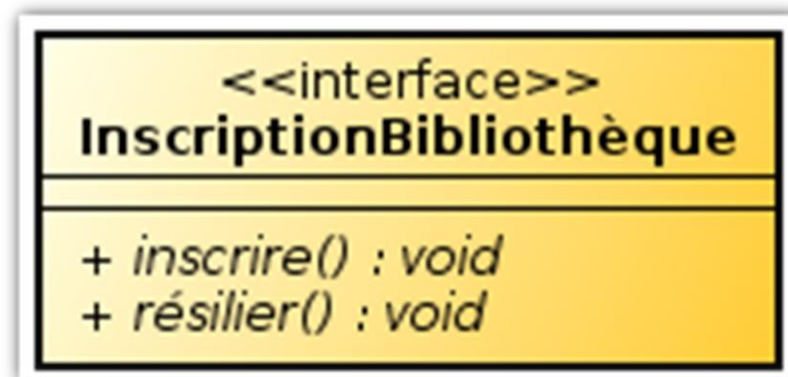
Le rôle de cette interface, stéréotypé <<interface>>, est de regrouper un ensemble d'opérations assurant un service cohérent.

Notion d'interfaces

Objectif : diminuer fortement le couplage entre deux classes.

Une interface ne dispose que de méthodes publiques abstraites (ou dit autrement, non définies).

La généralisation peut être utilisée entre interfaces, par contre, une interface ne peut être instanciée (comme une classe abstraite).



Relation de réalisation

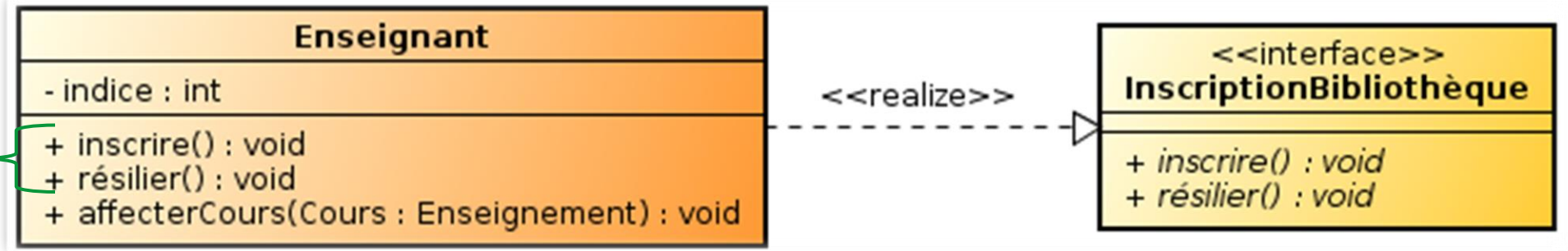
Comme l'interface n'a qu'un objectif de spécification, au moins un élément d'implémentation (une classe) doit lui être associée.

La relation de réalisation permet de mettre en relation une classe avec son implémentation.

Graphiquement, la réalisation se représente par un trait discontinu terminé par une flèche triangulaire parfois, mais pas nécessairement, stéréotypé <<realize>>

Relation de réalisation

Méthodes de
l'interface à définir



Attention : Une classe qui réalise (implémente) une interface est dans l'obligation de définir les méthodes décrites par l'interface (notion de contrat à respecter) comme lorsqu'une classe concrète redéfinit les méthodes abstraites héritées de sa classe parente abstraite.